Campus Resource & Activity Management System

Desktop GUI — User Guide

CS3400 Data Structures & Algorithms · Optional desktop edition · Qt 6 Widgets / C++17

Group member	Student number
M.E.C. Pama	201404452
L.V. Soai	202322577
R. Mphatle	201902287

> **What this is.** An optional Windows desktop interface for the CS3400 mini project. The graded submission remains the original **console application**; this GUI is an additional demonstration layer. It does **not** change any graded file — it links the project's own data-structure and algorithm source (linked list, AVL tree, hash table, FIFO queue, priority heap, stack/undo, graph, sorting, searching), unmodified, and simply presents them in a window.

1. Running the application (for the lecturer)

The application is delivered as a **portable folder**, `CS3400_Campus_Resource_Manager`. No installation, Qt or C++ compiler is required.

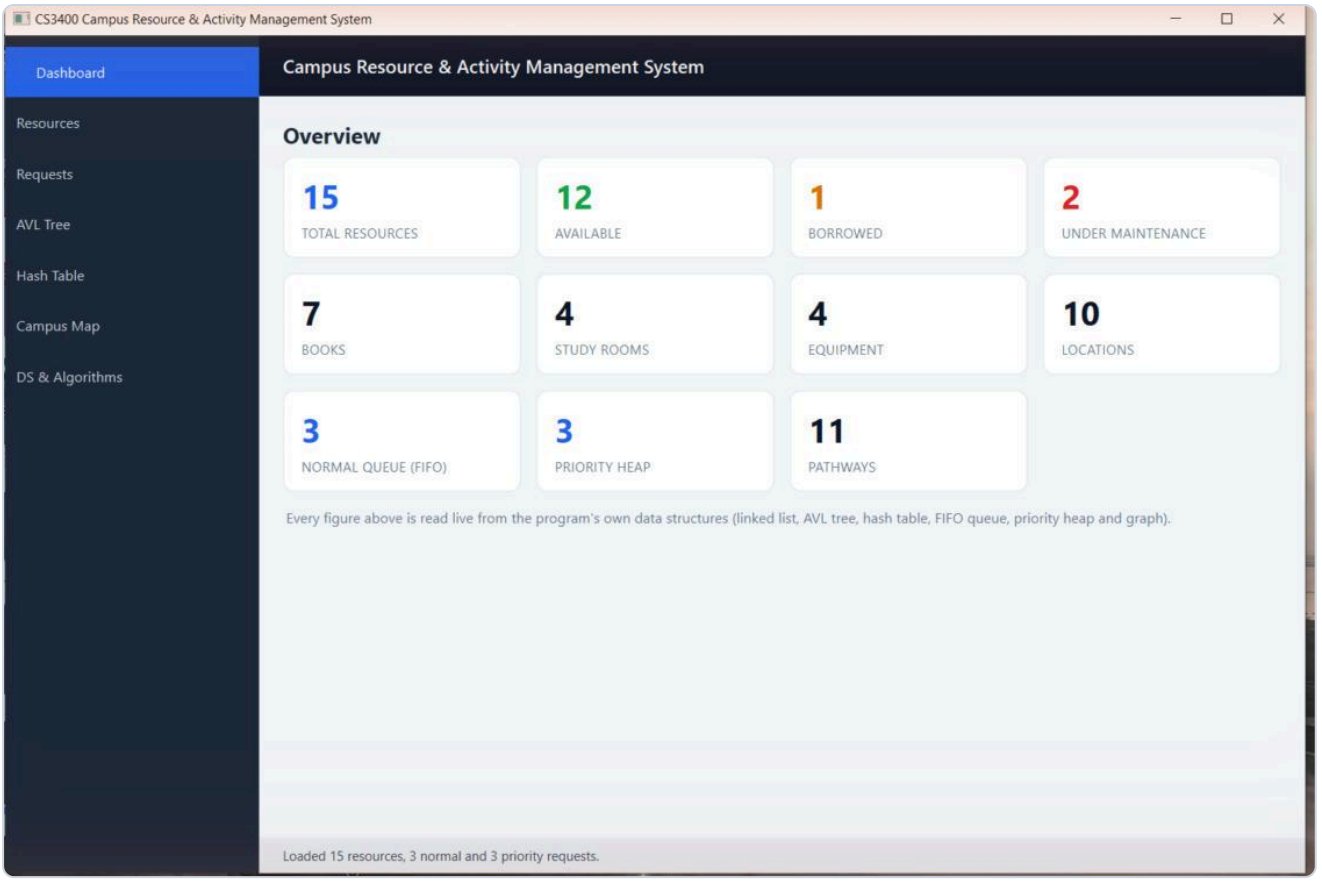
- Copy the whole folder onto any Windows PC (e.g. from a USB drive).
- Open the folder and double-click `CampusResourceManager.exe`.
- The application opens immediately. The Qt runtime DLLs, the `platforms\qwindows.dll` plugin and the MinGW runtime DLLs are bundled inside the folder, so nothing else is needed.

Where data is stored

The app reads and writes `resources.txt`, `requests.txt` and `locations.txt` in the `data\` sub-folder next to the executable, using the **same text format** as the console app. Use **Save to files** on the Resources screen to persist changes. The folder is self-contained and uses no absolute paths, so it works after being copied to another PC.

2. The screens

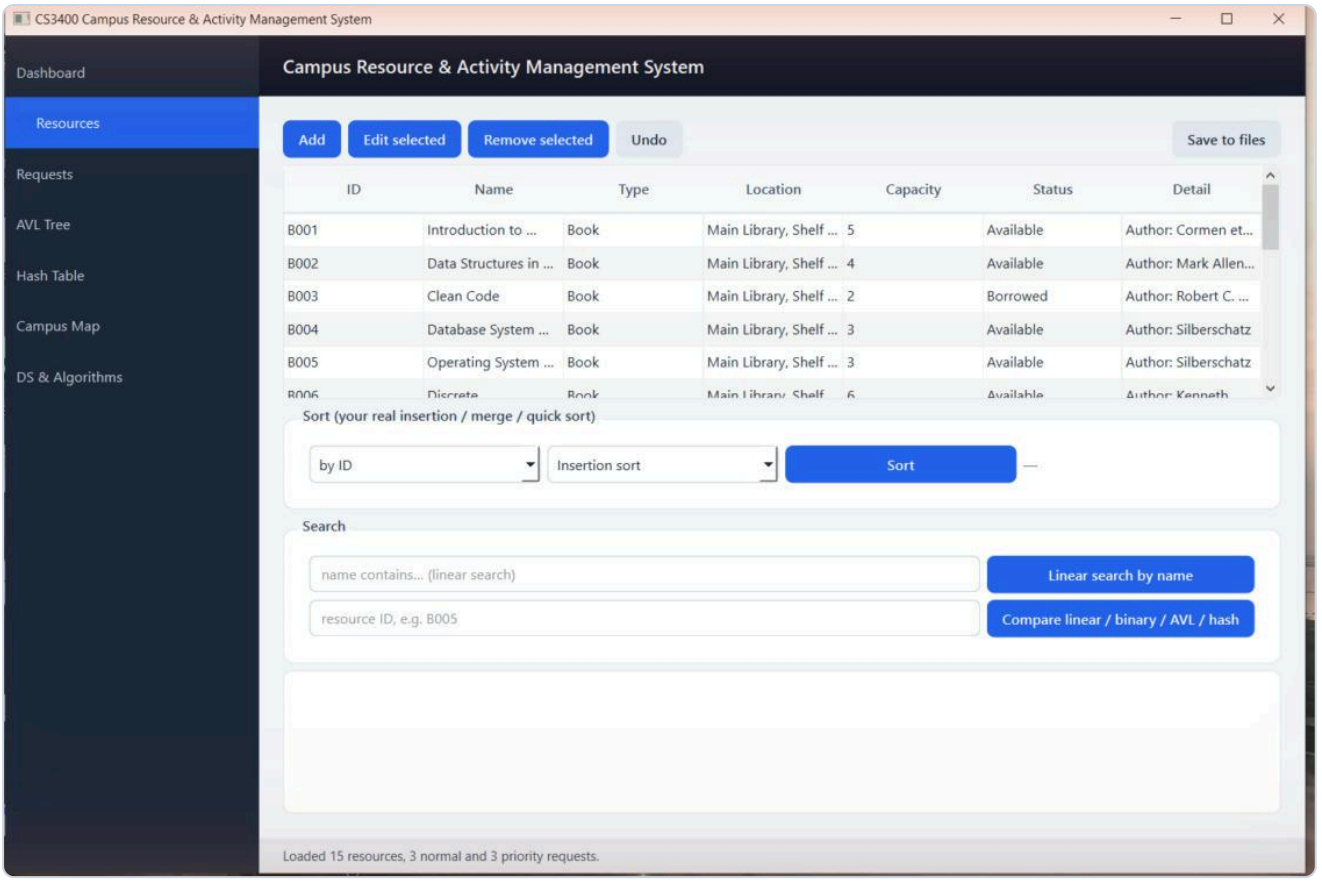
A left navigation sidebar switches between seven screens. Every value shown is read live from the program's real data structures at run time — nothing is hard-coded.



Dashboard — live counts from the linked list, AVL tree, hash table, FIFO queue, priority heap and graph.

Dashboard

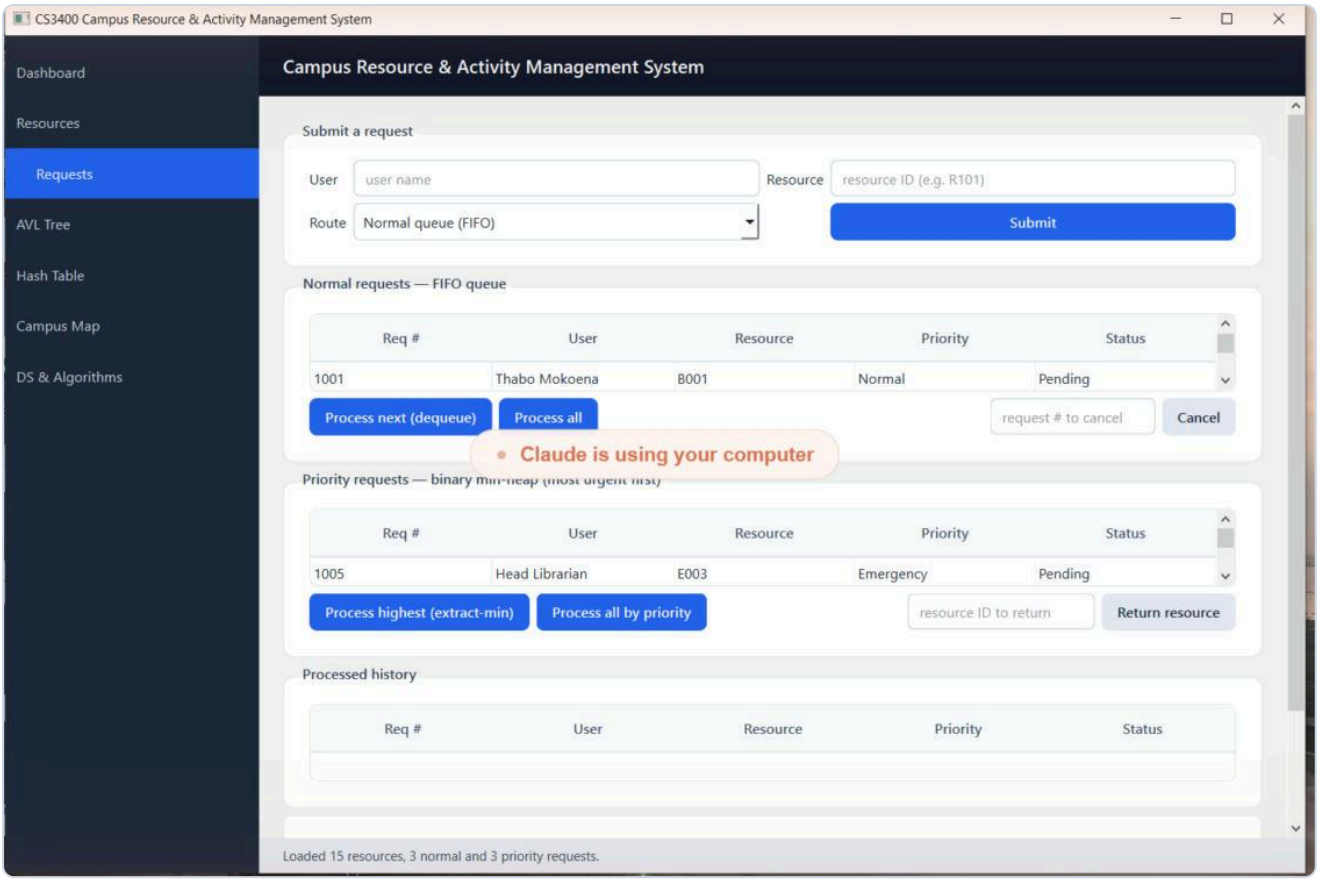
Totals by status (available / borrowed / maintenance), by type (book / study room / equipment), plus pending queue and heap sizes, location and pathway counts.



Resources — add / edit / remove / undo, sort (insertion / merge / quick), and search (linear / binary / AVL / hash).

Resources

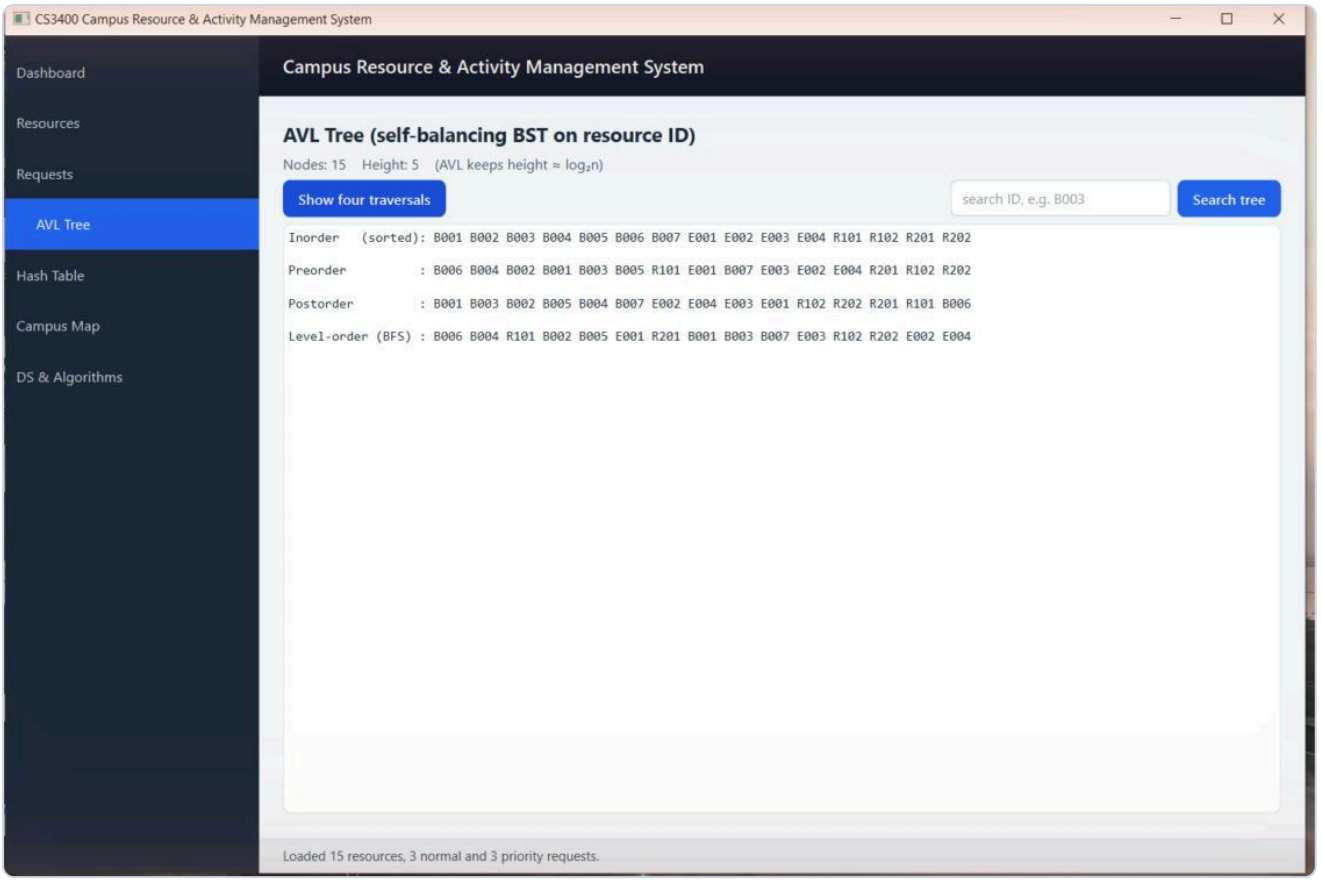
A table of all resources with controls to add, edit, remove and undo. **Sort** runs the project's real insertion, merge or quick sort by ID, name, capacity or type. **Search** runs a linear keyword search, or the four-way comparison (linear vs binary vs AVL vs hash) reporting the comparison count of each.



Requests — FIFO normal queue and priority min-heap, with process / cancel / return actions.

Requests

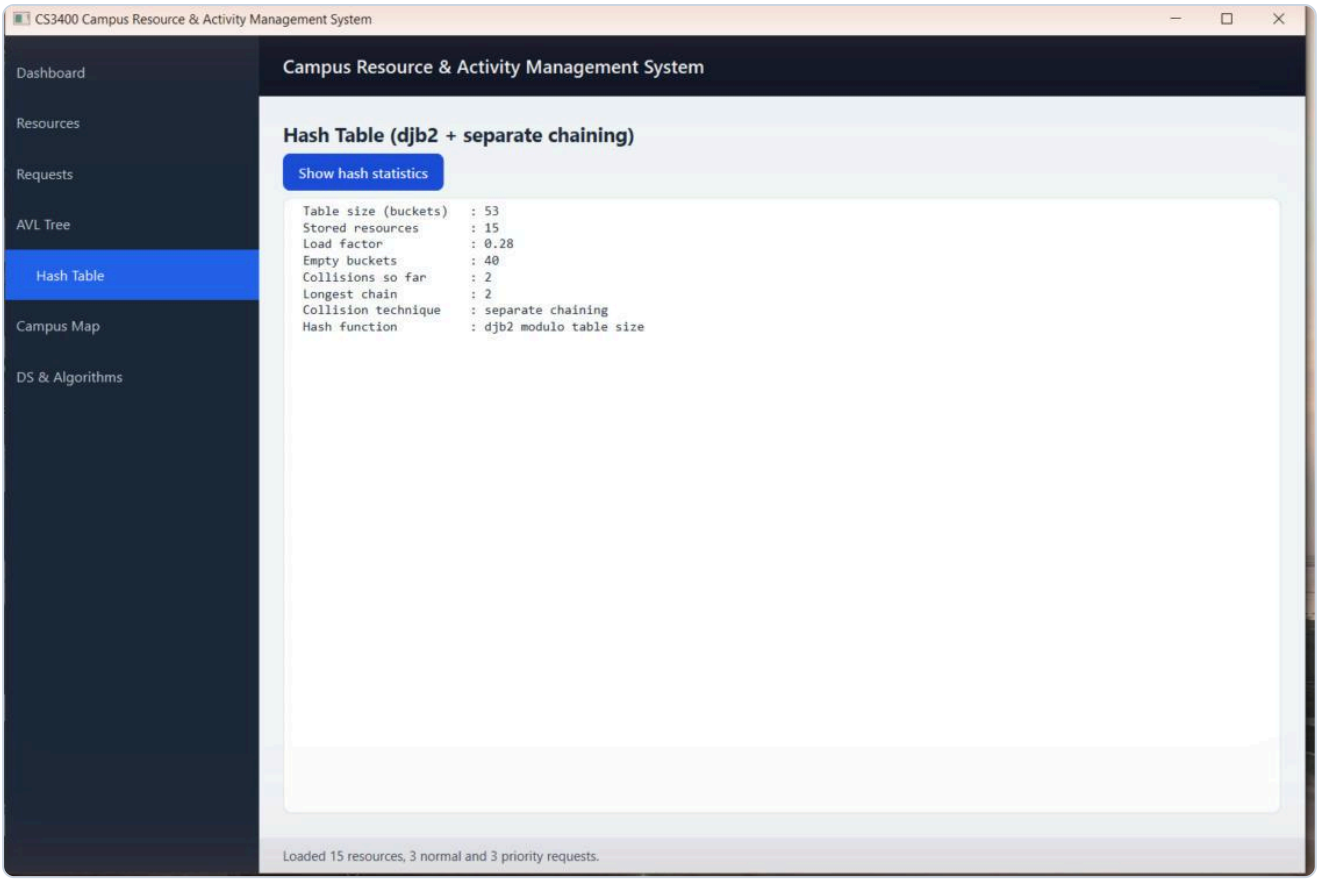
Normal requests are served first-in-first-out from the queue; priority requests are served most-urgent-first from the binary min-heap (Emergency < High < Normal < Low, ties by request ID). Processed requests move to the history table.



AVL Tree — the four traversals, node count and height, plus search by ID.

AVL Tree

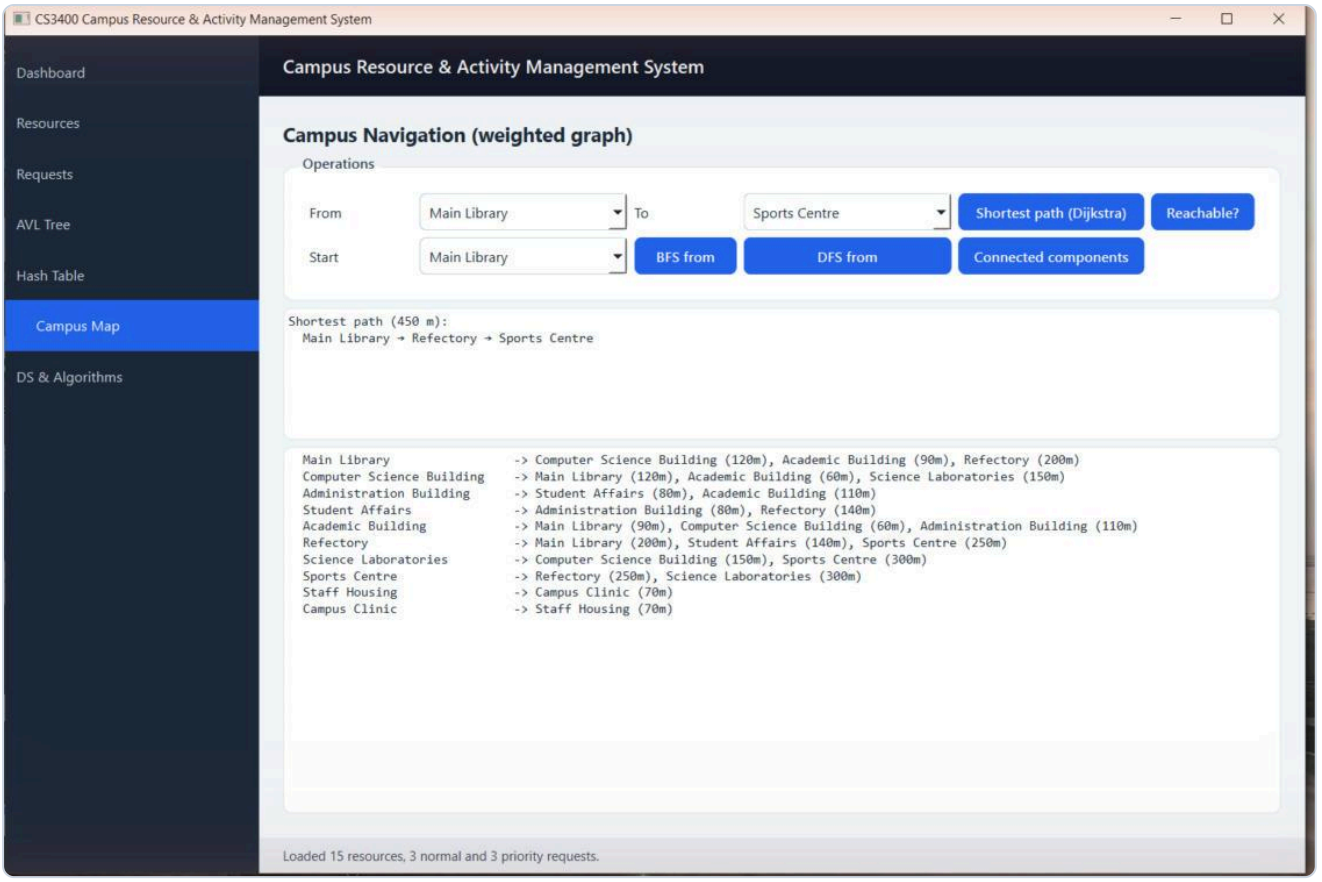
Shows the four traversals (in-order gives IDs in sorted order; pre-, post- and level-order), the node count and height, and a search that reports the comparison count.



Hash Table — djb2 + separate-chaining statistics.

Hash Table

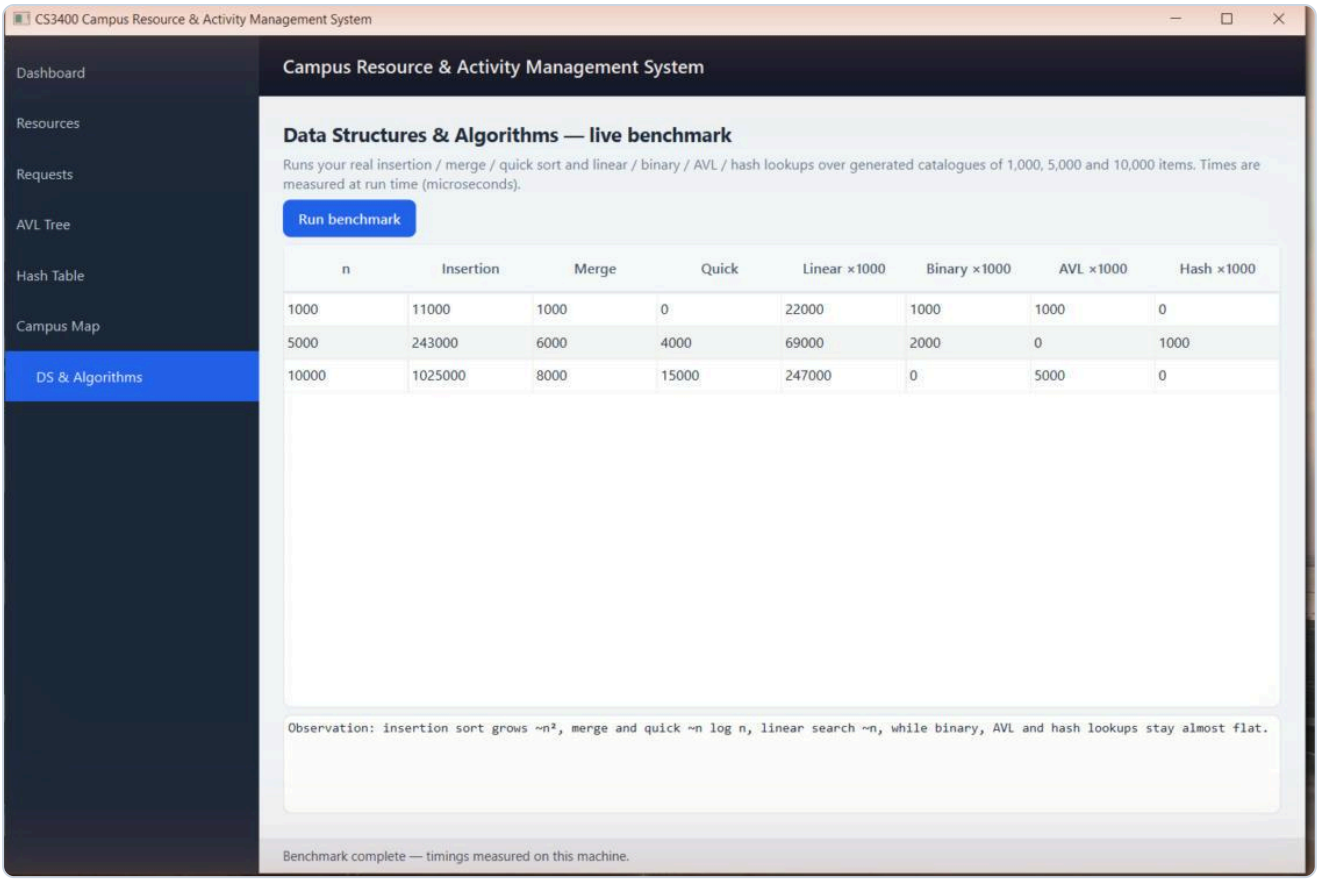
Reports table size, stored items, load factor, empty buckets, collisions and longest chain.



Campus Map — weighted graph with BFS, DFS, reachability, connected components and Dijkstra shortest path.

Campus Map

Lists the weighted adjacency structure and runs BFS, DFS, reachability, connected components and Dijkstra's shortest path between two chosen locations.



DS & Algorithms — a live benchmark of the real sorts and lookups at 1,000 / 5,000 / 10,000 items.

DS & Algorithms

Runs the real insertion / merge / quick sort and linear / binary / AVL / hash lookups over generated catalogues and reports the measured timings.

3. Building from source (optional)

The GUI source lives in the `gui\` folder and links the project's real `.cpp` files one level up. It does not compile `main.cpp` or `LibrarySystem.cpp` (those are console-coupled); a thin `GuiController` drives the same structures and the same data-file format.

- Prerequisite: Qt 6 with the MinGW kit (the `build.bat` provided will fetch Qt 6.8.1 + MinGW 13.1.0 automatically via `aqtinstall` if Qt is not installed).
- Double-click `gui\build.bat`. It compiles with `qmake` + `mingw32-make`, runs `windeployqt`, and produces the portable folder at `gui\release\CS3400_Campus_Resource_Manager`.

```
gui/
  campus_gui.pro   GuiController.h/.cpp   MainWindow.h/.cpp   main.cpp
  build.bat   README.txt   release/CS3400_Campus_Resource_Manager/ (the portable app)
```